Game Server & Client Programmer Portpolio

Autor : 박경준

E-Mail : pkj7323@gmail.com

HP : 010-4674-2738

PIP Project

프로젝트 개요

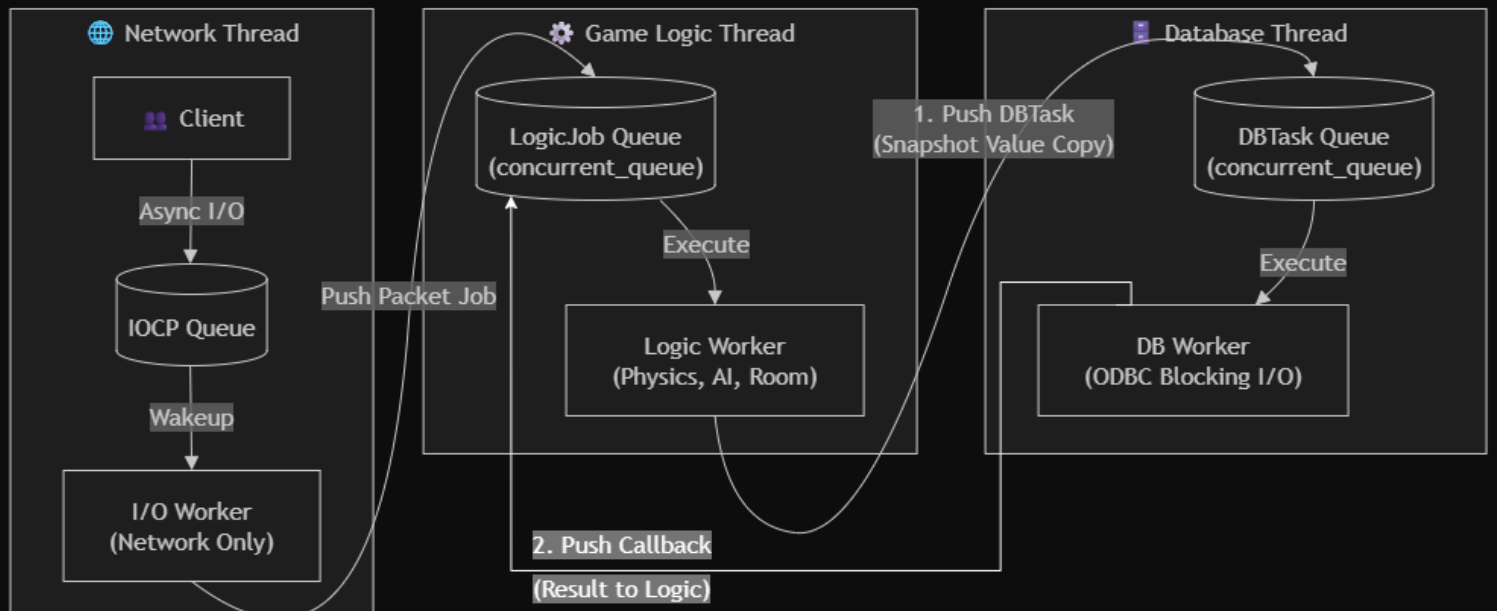
유튜브 영상 링크

프로젝트 이름	**PIP Project [S.T.L(Slay The Lord):구원자의 뒤틀린 웃음]**
장르	Action RPG (DirectX 12 + Jolt Physics)
설명	MO 액션 RPG
담당 역할	Server Physics & AI Sync, Client Engine Architecture
기술 스택	C++23, Jolt Physics, Behavior Tree, IOCP, DX12, Win32

개발 기간: 2025/08/01 ~ 2026/06/22, 개발 인원: 3인

PIP Server Architecture

서버 아키텍쳐 전체적인 설계도입니다.



Behavior Tree(BT)를 활용한 보스 AI

Problem: 보스의 복잡한 페이즈(Phase) 변화, 스킬, 타겟팅 로직이 얽혀 스파게티 코드 가능성

Solution: 행동 트리(BT)의 Action 노드를 상속 구조로 모듈화하고, 타겟(유저) 상실 시 즉시 대응하는 예외처리(안전장치) 노드를 Decorator로 배치

Performance Gain: 보스 패턴 로직의 재사용성 확보

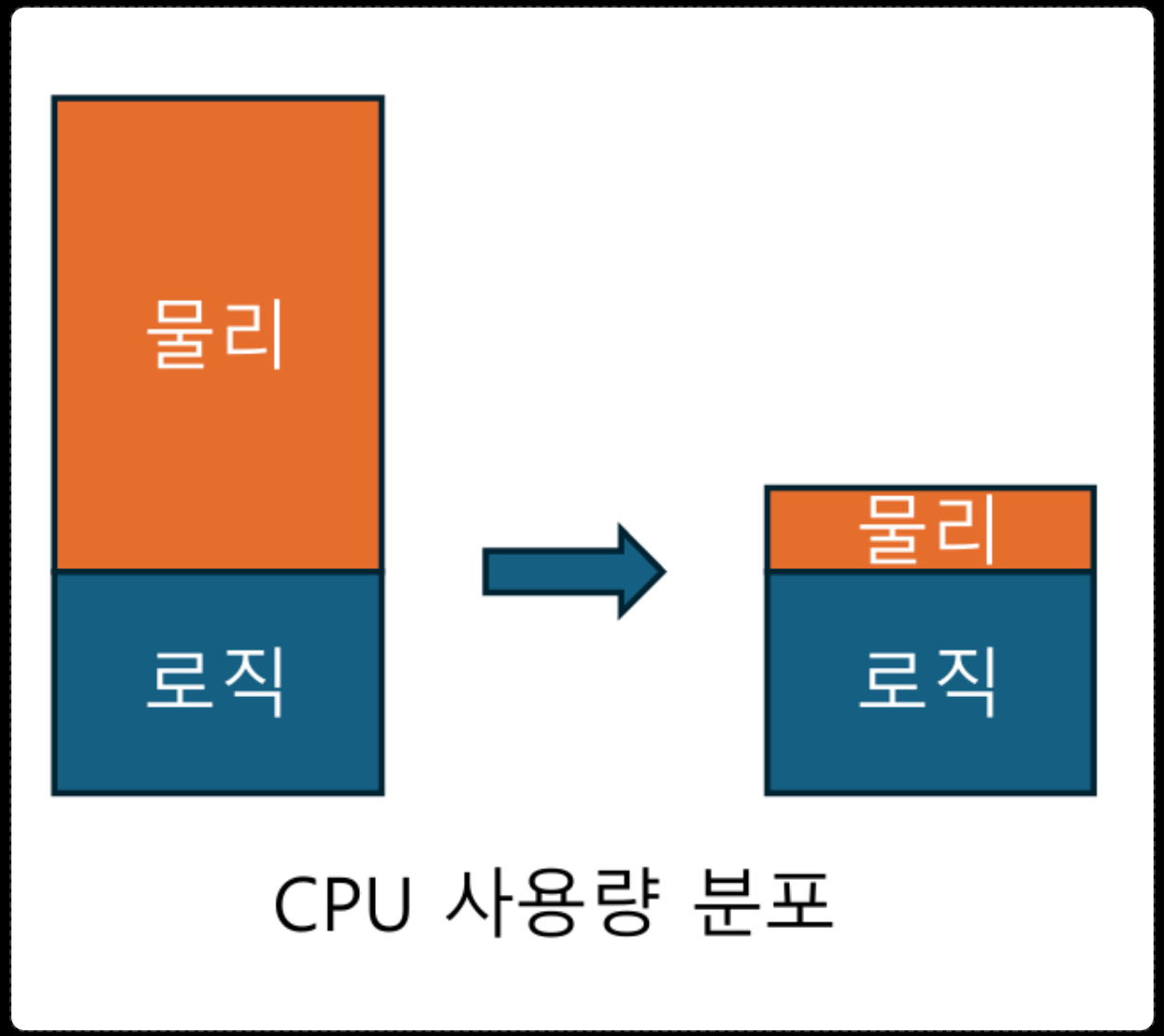
```
BTBuilder builder;
auto root = builder
    .sequence()
    .leaf<Condition_IsAlive>()
    .leaf<Condition_IsHitted>(DecoratorType::Inverter)
    .selector() // 전체 로직 우선순위 결정
        // --- [우선순위 1] 전투 로직 ---
        .sequence()
            .leaf<Condition_HasEnemy>() // 타겟(적군)이 있는가?
            .leaf<Condition_IsTargetInLeashRange>(25.0f) // 25m 내에 있으면 추격 유지
            .selector()
            // 1-1. 강력한 공격 시도 (사거리 2.5m)
                .sequence()
                    .leaf<Condition_IsEnemyInRange>(1.5f)
                    .leaf<Action_AttackEnemy>(heavyAtk)
                .end()
            // 1-2. 일반 공격 시도 (사거리 1.5m)
            .sequence()
                .leaf<Condition_IsEnemyInRange>(1.5f)
                .leaf<Action_AttackEnemy>(normalAtk)
            .end()
            // 1-3. 타겟 사거리 밖이면 네비메쉬 추격
            .sequence()
                .leaf<Action_UpdateEnemyPosToTarget>()
                .leaf<Action_FindPath>("MainStage_NavMesh")
                .leaf<Action_FollowPath>(6.0f) // 추격 속도 6.0
```

서버 물리 엔진(Jolt) 부하 억제 설계

Problem: 다수의 플레이어와 몬스터 객체 물리 연산을 서버가 모두 담당할 경우 심각한 물리 스레드 병목 발생

Solution: 대기 중인 NPC의 Motion Type을 Kinematic/Static(Sleep 상태)으로 설정하여 연산에서 제외하고, 교전 시 Dynamic으로 자동 승격(Dynamic Promotion)

Performance Gain: 충돌 연산 부하 최소화를 통한 안정적 서버 물리 프레임 타임 확보 (전체 CPU 사용량 84% -> 11%로 감소)



정밀 피격 동기화를 위한 지연 보상 (Rewind)

Problem: 네트워크 지연(Ping)으로 인해 클라이언트 화면의 몬스터 위치와 서버 위치가 달라 타격감 훼손

Solution: 모든 물리 객체의 최근 1초간 위치/히트박스 스냅샷을 원형 큐에 저장. 공격 패킷 타임스탬프 기반 과거 시간으로 Rewind 후 판정 처리

Performance Gain: Ping 100ms 환경에서도 불일치 없는 서버 피격 판정 달성

Data Race가 없는 안전한 Room(인스턴스 던전) 아키텍처

Problem: 한 방(Room)에 속한 객체 상태를 여러 워커 스레드가 동시 다발적으로 접근하여 수정하면 데이터 오염 및 데드락 위험

Solution: 방(Room)마다 고유한 Job Queue(명령 대기열)를 부여하여, 상태를 변경하는 모든 로직(이벤트)을 큐에 밀어넣고 단일 스레드 흐름으로 순차 처리(직렬화)

Performance Gain: 객체 단위의 동기화 락(Lock) 비용 완전 제거로 인해 서버 반응 속도 향상 및 데드락 100% 방지, MO RPG 게임 특성 활용

DB 스레드 분리 및 Stored Procedure 설계

Problem: 멀티플레이 환경에서 인벤토리를 영속화해야 했지만, ODBC 표준 API는 동기 블로킹 방식. 워커 스레드는 방 하나의 플레이어·몬스터·Jolt 물리 Step을 통째로 굴리기 때문에, 쿼리 한 번이 수십 ms를 잡아먹으면 안됨. 그렇다고 DB 스레드에 컴포넌트 포인터를 그냥 넘기면 데이터 레이스 발생

Solution: DB 전담 1개 신설. 작업 단위인 DBTask는 페이로드를 std::any로 담아 타입을 소거 — 저장은 로직 스레드가 인벤토리를 값 복사한 스냅샷을 make_any로 이동시켜 넣고, 로드는 반대로. 요청 시점의 logic_thread_idx를 사용, DB 스레드는 완료 콜백을 그 인덱스의 로직 큐로만 되돌려주고 로직 워커 스레드에서 싱글스레드 알고리즘으로 처리

좋은 점: 로직 스레드는 즉시 리턴하므로 DB 지연이 게임 틱에 영향 없음, 완료 콜백이원래 소유 스레드에서 실행되므로 Player·InventoryComponent 접근에 Mutex가 하나도 필요 없습니다. std::any 덕분에 새 DB 작업(스탯 등)을 추가해도 DBTask 구조체와 큐는 그대로 두고 핸들러만 등록하면 되며, 연결 문자열을 "DUMMY"로 주면

```cpp
// — 1) 스레드 구성 : IOCP I/O 2 + 로직(P코어-2) + DB 전담 1 ————————
auto p_cores = GetPerformanceCoreIndices();          // 하이브리드 CPU에서 P코어만 선별
int logic_count = std::max(1, (int)p_cores.size() - 2);
Server::Instance()->Start(/*io*/2, logic_count);      // DBManager는 내부에서 DB 스레드 1개 생성

// — 2) 작업 단위 : 페이로드를 std::any로 타입 소거 ————————————————
struct DBTask {
    DBTaskType    type;
    int           logic_thread_idx;  // ★ 콜백을 돌려받을 스레드 = 소유권 티켓
    DBJobCallback callback;          //   로직 스레드에서 실행될 함수
    std::any      data;              // ★ 타입 소거된 페이로드
};

// — 3) 저장 : 값 복사 스냅샷을 std::any로 이동 (공유 메모리 제거) ————————
InventorySnapshot snap;
snap.materials = inven->get_materials_snapshot();     // NRVO 값 복사
task.logic_thread_idx = _logic_thread_idx;
task.data = std::make_any<InventorySnapshot>(std::move(snap));
DBManager::Instance()->push_task(std::move(task));    // 논블로킹, 즉시 리턴

// — 4) 반환 : 완료 콜백을 '요청한 그 스레드'로만 push ————————————————
auto* q = Server::Instance()->get_logic_queue(task.logic_thread_idx);
q->push({ std::move(task.callback) });                // → any_cast 후 Mutex 없이 갱신
```

프로젝트 개요

프로젝트 이름	SImplest Minecraft MMO RPG GAME
장르	MMORPG
설명	IOCP 기반 C++ 멀티스레드 서버. 동접 4400명 및 단일 맵 20만 객체 동시 구동
담당 역할	Server Architecture & Optimization, Client Devlop
기술 스택	C++ IOCP ODBC MultiThreading Lua

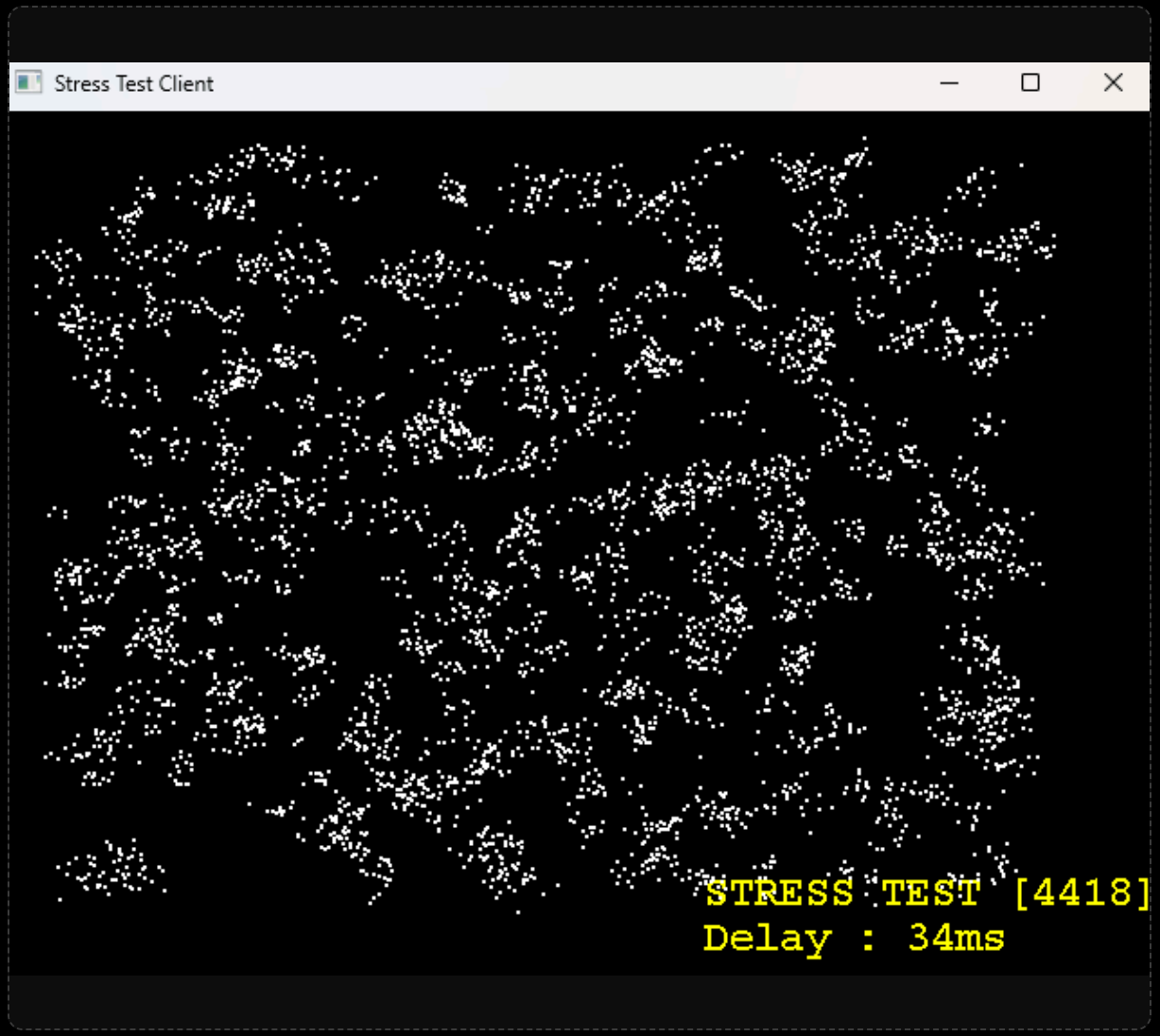
개발기간: 2026/05/04 ~ 2026/06/22
개발인원: 1인

시야 처리 병목 최적화 (Sector Grid)

Problem: 20만 마리의 NPC와 유저가 서로의 거리를 O(N^2)로 계산하여 서버 틱 심각한 저하 발생

Solution: 공간을 고정 크기의 Grid(Sector)로 분할하고 std::shared_mutex(Read-Write Lock)를 적용하여 인접 섹터의 객체만 탐색하도록 변경

Performance Gain: 탐색 복잡도를 O(k) 수준으로 낮추어 동시 접속자 수 4400명 안정적 유지



네트워크 I/O 병목 및 데이터 프래그멘테이션 최적화

Problem: 매 틱마다 쏟아지는 수많은 객체의 이동/투사체 패킷 전송(Send) 시스템 콜 비용 과다

Solution: 전송 큐에 패킷을 버퍼링 후 WSABUF 배열을 이용한 Scatter-Gather I/O(Batching)로 뭉쳐서 보내고, 수신단에서 링 버퍼 기반 재조립(Reassembly) 파서 구현

Performance Gain: WSASend 시스템 콜 횟수 대폭 감소 및 패킷 유실률 제로 달성

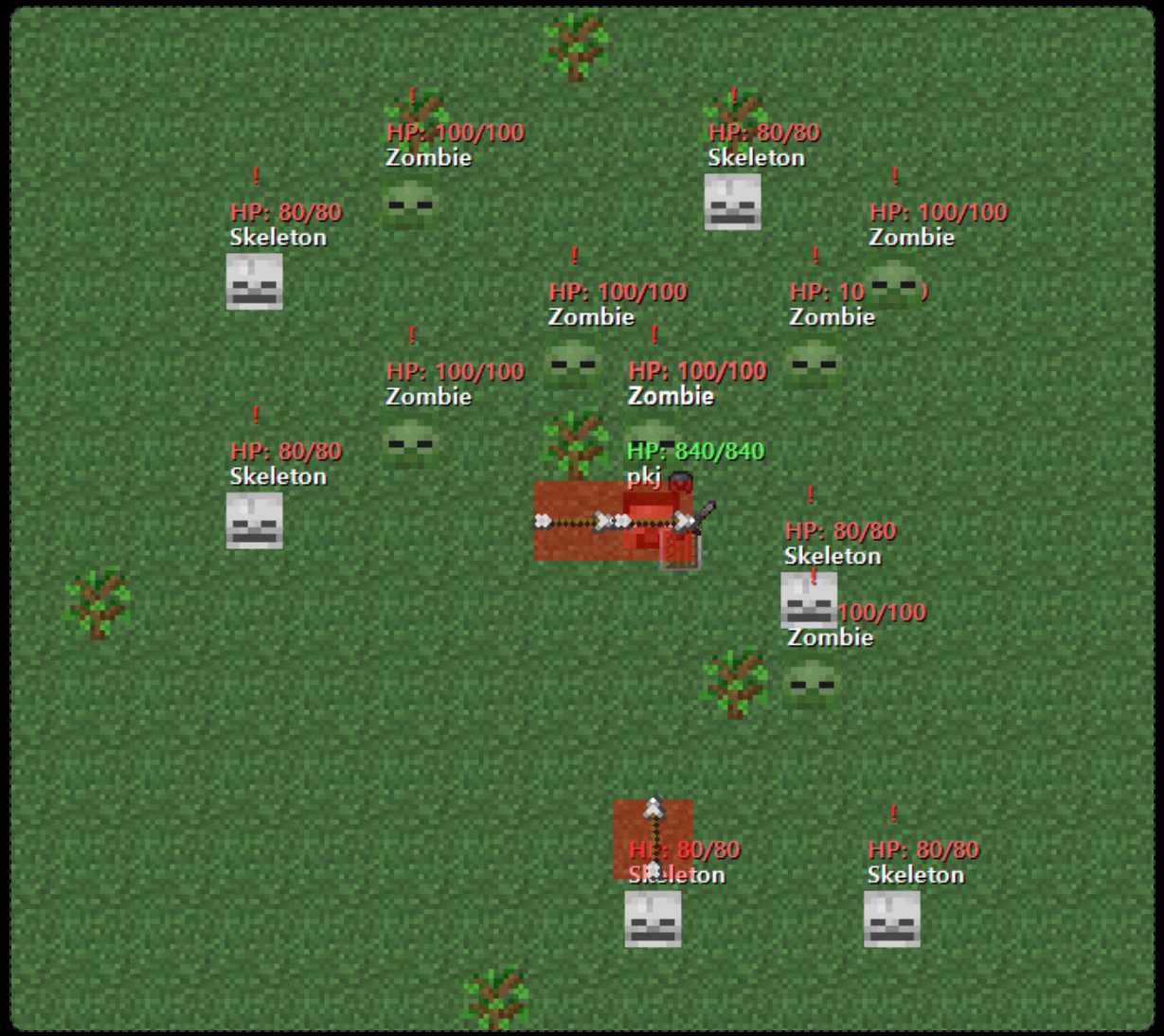
```cpp
// 1. 송신 버퍼링 및 Scatter-Gather용 WSABUF 배열 세팅
EXP_OVER(std::vector<std::vector<char>>&& data)
    : type(io_type::send), pending_data(std::move(data))
{
    ZeroMemory(&over, sizeof(over));
    wsa_bufs.reserve(pending_data.size());

    // 버퍼 복사 없이 포인터와 크기만 배열로 엮기 (Scatter-Gather)
    for (auto& d : pending_data) {
        wsa_bufs.push_back({ static_cast<ULONG>(d.size()), d.data() });
    }
}

// 2. 비동기 락프리 기반 전송 (Flush)
void SESSION::flush_send() {
    std::vector<std::vector<char>> sending_data;
    {
        std::lock_guard<std::mutex> lock(send_mtx);
        sending_data.swap(send_queue); // 큐에 쌓인 패킷들 한 번에 들고오기
    }

    EXP_OVER* send_over = new EXP_OVER(std::move(sending_data));
    DWORD send_bytes;

    // 모아둔 WSABUF 배열을 한 번의 시스템 콜로 일괄 전송
    WSASend(client_, send_over->wsa_bufs.data(),
            static_cast<DWORD>(send_over->wsa_bufs.size()),
            &send_bytes, 0, &send_over->over, NULL);
}
```

Lua Script를 활용한 확장성 있는 NPC AI

Problem: 20만 객체의 행동 패턴을 C++에 하드코딩하면 밸런스 한 줄을 고칠 때마다 재컴파일·재배포가 필요합니다. 그렇다고 Lua를 붙이면 lua_State가 재진입 불가라, 전역 하나를 공유하는 순간 모든 워커 스레드가 AI 호출마다 뮤텍스를 두고 경쟁해서 성능이 떨어집니다.

Solution: thread_local lua_State 풀을 두어 워커 스레드마다 독립적인 VM 7개를 보유하게 했습니다. 이를 가능하게 한 핵심 설계는 AI 스크립트를 순수 함수로 강제한 것입니다 — NPC의 모든 상태(HP·좌표·상태·타겟)는 C++가 소유하고 매 호출 ctx 테이블로 주입, 결과는 new_state·action으로만 반환받습니다. 보스 페이즈 전환과 크리퍼 자폭 카운트다운까지 이 왕복으로 처리해, lua_State를 상태 저장소가 아닌 순수 실행 엔진으로 만들었습니다. (저장,실행은 C++, 판단만 lua)

Result: AI 호출 시 락 획득 0회, lua_State 개수 = 스크립트 7종 × 코어 수 (16코어 기준 112개) — NPC가 20만이든 100만이든 불변 · NPC 이벤트를 IOCP로 아무 워커나 처리 — 스레드 핀닝·라우팅 로직 불필요, 부하 분산이 자동 · 한 스레드의 Lua 에러가 lua_pcall에 격리되어 다른 스레드로 전파되지 않음 · GC가 State 단위로 분산되어 전역 방식의



Lua Script를 활용한 효율적인 NPC AI

1. 구현 구조

상태 저장소가 두 개로 갈려 있습니다.

```
// npc_ai_manager.h
// (A) 전역 1개 – 경로만 보관
static std::unordered_map<int, std::string>& script_paths() {
    static std::unordered_map<int, std::string> paths;   return paths;
}
// (B) 스레드마다 1개 – 실제 VM 보관
static std::unordered_map<int, lua_State*>& get_pool() {
    thread_local std::unordered_map<int, lua_State*> pool;   return pool;
}
```

```
// main() – 경로만 등록 (실제 로드는 각 스레드가)
const std::string ai_dir = "ai/";
NpcAiManager::register_script(1, ai_dir + "zombie_ai.lua");
NpcAiManager::register_script(2, ai_dir + "skeleton_ai.lua");
// ... creeper, enderman, iron_golem, ender_dragon, fire_zone

// worker_thread() 진입부 – 이 스레드 전용 VM 7개 생성
NpcAiManager::init_thread_states();
```

```
static void init_thread_states() {
    auto& pool = get_pool();
    for (auto& [type_id, path] : script_paths()) {
        if (pool.contains(type_id)) continue;
        lua_State* L = luaL_newstate();
        luaL_openlibs(L);
        if (luaL_dofile(L, path.c_str()) != LUA_OK) { /* 로그 후 lua_close */ continue; }
        pool[type_id] = L;
    }
}
```

호출 경로:

```
NPC 타이머 만료 → PostQueuedCompletionStatus(h_iocp, -1, obj_id, &move_over->over)
                → 아무 워커 스레드가 수신 (io_type::npc_move)
                → NpcAiManager::call_on_timer(npcType, ctx)
                → call_hook: lua_getglobal → ctx 테이블 push → lua_pcall → 반환 테이블 파싱
                → NpcAction (MOVE / MELEE / AOE / BREATH / DIE ...) 을 C++가 실행
```

혹은 3개: `on_timer(ctx)`, `on_hit(ctx, attacker_id, damage)`, `on_spawn(ctx)`.

2. 이 설계가 성립하는 전제 — 스크립트가 순수 함수다

`ai/` 7개 스크립트를 전부 확인한 결과, **Lua 쪽에 NPC별 기억이 하나도 없습니다.** 모든 가변 상태(hp, 좌표, 상태, 타겟)는 C++가 소유하고 매 호출마다 `ctx`로 주입되며, 결과는 반환 테이블로만 나옵니다.

```
-- ender_dragon_ai.lua : 보스 페이즈 전환조차 ctx에서 파생
local hp_percent = ctx.self_hp / ctx.self_max_hp
if hp_percent <= 0.50 and ctx.current_state == 2 then
    return { action = CHAT, new_state = 3, chat = "크롸롸롸! (2페이즈 돌입)", delay_ms = 2000 }
end
```

```
-- creeper_ai.lua : 자폭 카운트다운도 상태를 Lua에 두지 않고 new_state로 왕복
if ctx.dist <= 1 then
    return { action = ACTION_DIE, target_id = ctx.target_id, new_state = STATE_FLEEING,
             chat = "쉬이이이...", delay_ms = 3000 }
end
```

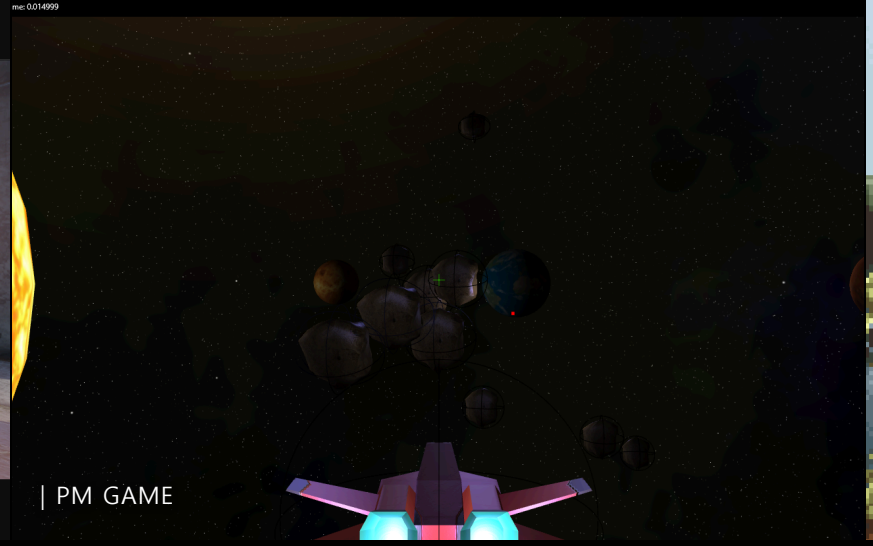
DataBase 서버 연동

Problem: 멀티플레이 게임에서 유저의 로그인 정보와 진행 상황(좌표·레벨·장비·인벤토리·퀘스트·스킬)을 영속화할 저장 시스템이 필요했습니다. 그런데 ODBC 표준 API는 동기 블로킹이라, IOCP 워커 스레드에서 직접 호출하면 쿼리가 끝날 때까지 그 스레드가 통째로 묶여 다른 유저의 패킷 처리까지 정지합니다. 로그인 한 번에 조회가 5번 필요한 구조라 접속이 몰리면 워커 풀이 전부 DB 대기로 빠지는 기아 상태가 발생합니다.

Solution: Windows가 ODBC 표준 API를 지원한다는 점을 활용해 MSSQL + Management Studio로 스키마와 저장 프로시저를 직접 설계하고, ODBC 핸들을 DB 전용 접근 스레드에 배치했습니다. 워커 스레드는 DBTask를 lock-free 큐에 던지고 즉시 복귀하며, 페이로드는 std::variant<monostate, string, player_data>로 담아 요청과 응답이 같은 객체로 왕복하게 했습니다. 완료 통지는 별도 채널을 만들지 않고 PostQueuedCompletionStatus로 IOCP에 되돌려, DB 결과가 일반 네트워크 I/O 완료와 동일한 경로로 배달되게 했습니다.

좋은 점: 메인 로직·I/O 스레드와 별개의 스레드에서 비동기로 연산이 처리되므로 DB가 아무리 느려도 게임 로직은 계속 돌아갑니다. 완료 통지에 IOCP를 재사용한 덕분에 콜백 큐·디스패처 없이 기존 io_type switch에 case 하나만 추가하면 됐습니다. 또한 모든 쿼리를 저장 프로시저로 캡슐화해 C++ 코드에 테이블·컬럼명이 전혀 노출되지 않고, 스키마가 바뀌어도 SP만 수정하면 서버 재컴파일이 불필요하며, DB 계정에 EXECUTE 권한만 부여할 수 있어 안전한 영속성 저장 체계를 만들었습니다. 종료 시에는 큐에 남은 작업을 모두 비우는 Drain 루프로 마지막 저장 유실도 막았습니다.

```cpp
// ── 1) 작업 단위 : 요청·응답을 variant 하나로 왕복 ──────────────────
using DBTaskData = std::variant<std::monostate, std::string, player_data>;
struct DBTask {
    int        ioType;      // 완료 시 IOCP에 실어 보낼 io_type
    DBTaskType type;        // LOGIN_AUTH / SAVE_USER_DATA
    int32_t    session_id;
    DBTaskData data;        // 요청=user_id(string) → 결과=player_data 로 제자리 교체
};

// ── 2) 완료 통지 : 별도 채널 없이 IOCP를 재사용 ──────────────────────
DBManager::Instance().Init([](int session_id, int io_type_val, const DBTask& task) {
    EXP_OVER* over = new EXP_OVER;
    over->type    = static_cast<io_type>(io_type_val);
    over->db_task = task;                                   // 결과를 확장 OVERLAPPED에 적재
    PostQueuedCompletionStatus(h_iocp, 1, session_id, &over->over);   // 가짜 I/O 완료 이벤트
});
DBManager::Instance().StartDBThread();

// ── 3) DB 전담 스레드 : 핸들러 맵 디스패치 + 종료 시 Drain ──────────
while (m_db_thread_running) {
    DBTask task;
    if (m_task_queue.try_pop(task)) {                   // PPL lock-free 큐
        auto it = m_handlers.find(task.type);
        if (it != m_handlers.end()) it->second(task);     // ODBC 블로킹은 여기서만
        if (m_notifyCb) m_notifyCb(task.session_id, task.ioType, task);
    }
    else std::this_thread::sleep_for(std::chrono::milliseconds(10));
}
while (m_task_queue.try_pop(task)) { ... }              // 종료 전 잔여 작업 전량 처리

// ── 4) 워커 스레드 : 기존 switch에 case 하나 추가로 끝 ──────────────
case io_type::db_player_auth: {
    player_data result = std::get<player_data>(o->db_task.data);
    if (result.success) new_session->player_->x_ = result.x;   // 로드 데이터 주입 후 입장
}
```



PROJECTS

게임 엔진 프로젝트 unity 개발 프로젝트

- unity 기능을 최대한 활용한 게임 개발
- 스플라인, 프리팹, 콜리전, 콜라이더, Scene Hirachy

PM GAME 컴퓨터 그래픽스

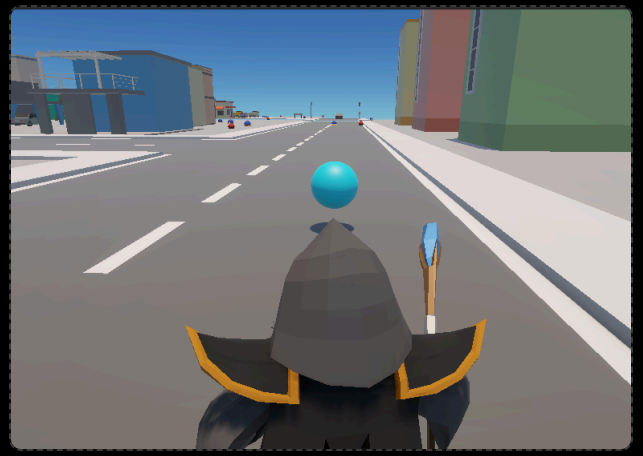
- 컴퓨터 그래픽스 및 OpenGL 사용한 게임개발
- 고급 텍스쳐링 기법 사용, 직접광, 스포트 라이트, 점 조명 구현
- 씬, 메인루프, 입력, 오브젝트 생성 삭제, 그래픽스 엔진, glsl, 기본적인 3D 자체 게임 엔진

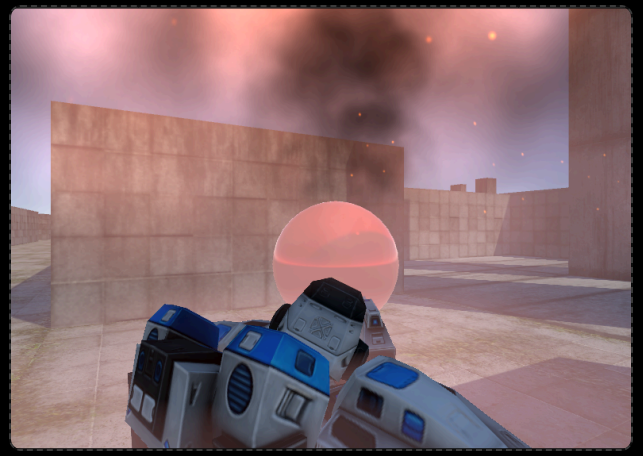
메탈 슬러그 모작 windows programming

- win32사용한 2D 게임 개발
- 충돌체와 이동 속도 계산을 위한 정밀한 물리 도입
- 2D 애니메이션을 위한 세밀한 상태머신 개발

Unity Project

프로젝트 개요



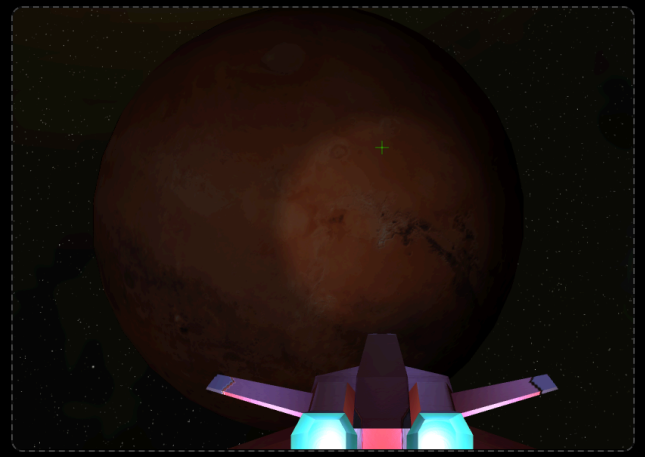


프로젝트 이름	Unity Project
장르	TPS,레이싱 등
설명	유니티 6.0을 사용한 여러 프로젝트 (5개)입니다
개발 인원	1명
담당 역할	클라이언트 개발
개발 언어	untiy6, c#
사용 툴 & 라이브러리	Visual Studio, Git, unity
제작 기간	2025 / 05 ~ 2025 / 06 (한달)

유튜브 링크

PM GAME | 3D Space Shooting Game

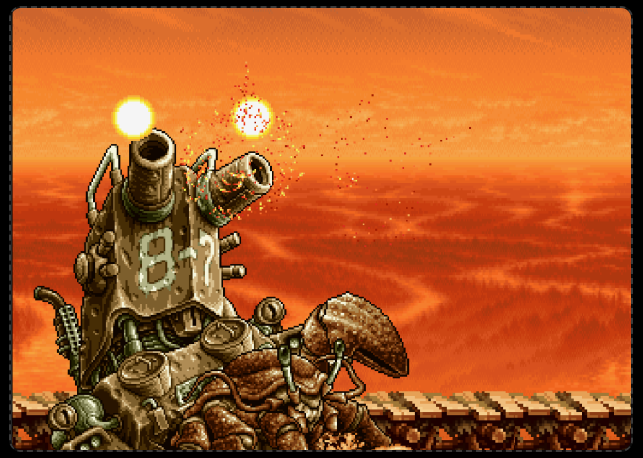
프로젝트 개요



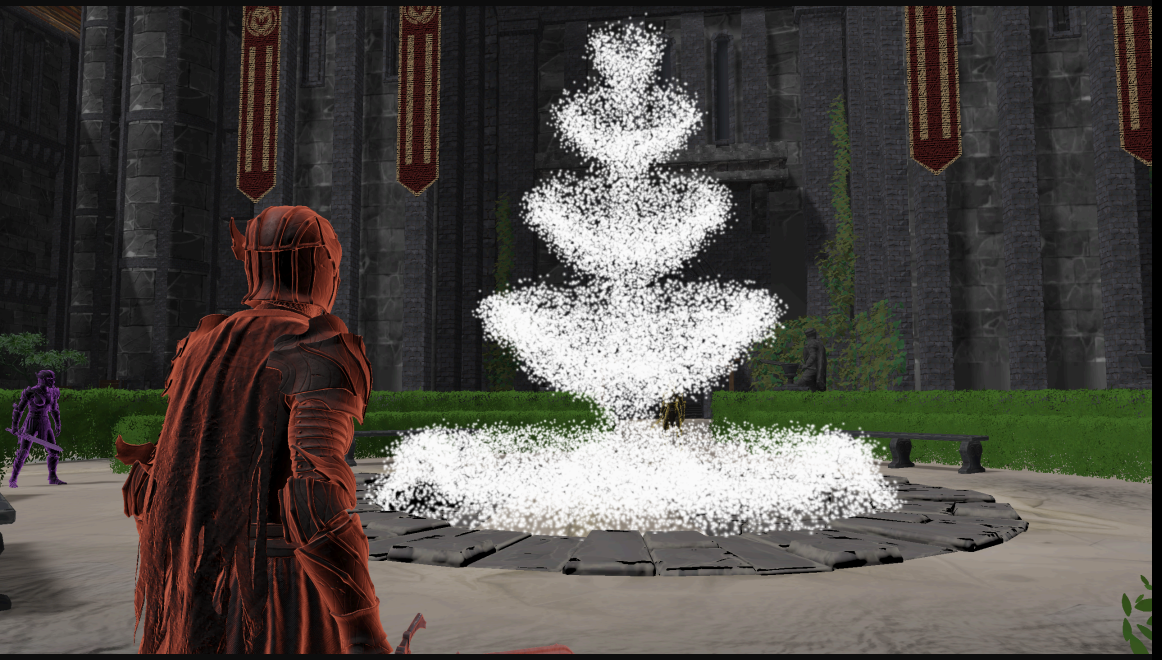
프로젝트 이름	PM GAME
장르	3D 1/3인칭 슈팅게임
설명	3D 1/3인칭 우주선 슈팅 게임, 날아오는 운석을 파괴하라
개발 인원	2명
담당 역할	그래픽스, 오브젝트 관리, 게임 메인 루프, 입력, 사운드, 조명, 적 로직, 플레이어 로직 등 거의 모든 클라이언트 개발 및 자체 엔진 개발 제작
개발 언어	C++20
사용 툴 & 라이브러리	Visual Studio, Git, OpenGL, GLM, fmod
제작 기간	2024 / 10 ~ 2024 / 11

Metal Slug 모작

프로젝트 개요



프로젝트 이름	메탈슬러그 모작
장르	2D 플랫포머 슈팅
설명	메탈슬러그 게임을 win32라이브러리를 사용해서 모작을 만들었습니다.
개발 인원	2명
담당 역할	애니메이션 스프라이트 편집, 상태머신 작성, 플레이어 로직, 보스와 적의 공격 로직 모든 리소스 관리,편집
기술 스택	C++20, win32, fmod
사용 툴 & 라이브러리	Visual Studio, Git
제작 기간	2024 / 05 ~ 2024 / 06



Thank You

포트폴리오를 검토해 주셔서 감사합니다.

Author : 박경준

E-Mail : pkj7323@gmail.com

HP : 010-4674-2738